Episodio 12 de 14: Juicio a la réplica (1): escala, método y Folly

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 11, «El veredicto y los límites».
- **Episodio siguiente:** episodio 13, «Juicio a la réplica (2): corrección, RAM, estadística y sentencia».

Lo esencial de este episodio:

- En el lenguaje de la ACM, esto es una reproducción: otro equipo, con los artefactos de los autores.
- Cargos 1 a 7: la escala sin 4 sockets, la simulación de sockets, los recortes, Folly, la latencia ruidosa, Docker y los propios errores.
- Lo más fuerte de la defensa: la diferencia de Folly ya existe con un hilo, y Docker, los recortes y la distancia se midieron en vez de suponerse.

---

2. Antes del debate: ¿réplica o reproducción?

Un juez exigente puede empezar por aquí, así que conviene tenerlo claro.

La ACM, la asociación que pone las insignias de reproducibilidad en muchos congresos, distingue tres palabras:

Palabra	Quién lo hace	Con qué	¿Es lo nuestro?
Repetición	El mismo equipo	Su propio montaje	No
Reproducción	Otro equipo	Los artefactos de los autores	**Sí**: programas oficiales del paper, en otra máquina
Réplica	Otro equipo	Artefactos hechos por él, desde cero	**En parte**: `bench.cpp` es un programa de pruebas del equipo, pero usa la implementación de los autores

Qué hacer con esto: en el curso se le llama «réplica», y está bien. Pero si un juez dice «esto es una reproducción», **dale la razón**. No cuesta nada y demuestra que sabes de qué hablas. Y añade que, además, `bench.cpp` aporta una segunda opinión con un programa distinto.

---

3. El tribunal: 14 cargos contra la réplica

Cómo funciona. Hay tres voces, y las tres las hago yo:

- **Juez escéptico.** Busca el punto débil y aprieta. Es el jurado más duro que te puedes encontrar.
- **Juez defensor.** Responde solo con datos que estén en el repositorio.
- **Presidencia.** Escucha a los dos y dicta un fallo:
  - **A favor de la réplica**: la crítica no se sostiene.
  - **Concedido en parte**: la crítica tiene algo de razón, y hay que reconocerlo.
  - **En contra**: la crítica tiene razón, y hay que arreglarlo.

Cada cargo termina con **cómo blindarlo** (qué mejorar) y una **respuesta de 20 segundos** para decirla tal cual.

El reparto sugerido sigue a RESPONSABILIDADES.md: corrección y latencia para David; rendimiento para Jheremy; memoria y comparación para Diego. Ajústalo a quien exponga cada parte.

#	Cargo	Fallo	Quién lo defiende
1	«Sin 4 sockets no es una réplica»	Concedido en parte	Jheremy
2	«Vuestra simulación de sockets es un decorado»	A favor, con matiz	Jheremy
3	«Recortasteis las mediciones»	A favor, con matiz	Jheremy
4	«Folly no se reproduce porque lo hicisteis mal»	A favor; la pregunta sigue abierta	Jheremy
5	«La latencia es puro ruido»	Concedido en parte	David
6	«Docker contamina las medidas»	A favor	Diego
7	«Os equivocasteis tres veces: ¿por qué fiarse?»	A favor	Quien lo cuente
8	«“0 errores” no demuestra que sea correcto»	Concedido, y ya lo decís	David
9	«Con un millón de pilas solo medisteis vuestra RAM»	Concedido en parte	Diego
10	«Elegisteis lo que os convenía»	A favor	Cualquiera
11	«Sin intervalos de confianza no hay ciencia»	Concedido en parte	Jheremy
12	«La máquina no estaba controlada»	Concedido en parte	Cualquiera
13	«¿Esto lo hicisteis vosotros o una IA?»	Depende de vosotros	Todos
14	«Vuestro informe contradice vuestros	**En contra, si no se corrige antes**	Cada responsable

#	Cargo	Fallo	Quién lo defiende
	datos»		

Cargo 1. «Sin 4 sockets no es una réplica»

Juez escéptico: El paper mide la figura 5 en 4 Xeon E7-8867: 72 núcleos y 144 hilos, con la memoria repartida entre 4 chips. Vosotros tenéis un PC de escritorio con 24 núcleos, un solo chip y sin NUMA. Habéis medido otra cosa.

Juez defensor: Hemos medido **lo mismo, a otra escala**, y lo decimos. Tres argumentos:

1. **Se comparan algoritmos, no máquinas.** Esta CPU es casi diez años más moderna, así que los Mop/s absolutos no significan nada. Lo que se compara es el **orden** y la **proporción**: si Autores era el doble de rápido que un rival allí, ¿lo es también aquí? (Parte 6, regla 1.)
2. **La escala se conservó.** Los hilos del paper (1 a 216) se dividieron entre 6: 24 hilos es «toda la máquina», como 144 allí, y 36 es 1,5 veces, como 216 (Parte 4.3).
3. **Lo que depende de la escala está marcado.** Por ejemplo, en el paper Anderson solo adelanta a Autores con n = 1 y u = 1 % **a partir de 54 hilos**. Aquí no se llega a esa zona, y el documento lo explica como «una diferencia de alcance más que una contradicción» (Parte 6.1).

Juez escéptico: Entonces no podéis decir nada sobre NUMA ni sobre 4 sockets.

Juez defensor: Correcto, y no lo decimos. Está en la Parte 8, «No hay 4 sockets» y «No hay NUMA»: `numactl -i all` aquí no hace nada.

Presidencia: concedido en parte. La reproducción vale para comparar algoritmos a escala de un chip. Cualquier conclusión sobre 4 sockets o NUMA queda fuera, y el documento ya lo reconoce. No es un defecto escondido: es un límite declarado.

Cómo blindarlo: - Lo ideal: repetir `correr_docker.sh` en una máquina de 2 o 4 sockets, sea un servidor de la universidad o una instancia *bare-metal* en la nube. Antes hay que medir su mapa con `herramientas/latencia_nucleos.cpp` y adaptar las listas de núcleos de `baterias.py`. - Lo barato: repetir la batería de afinidad en un **AMD Ryzen con dos chiplets (CCD)**, si alguien del equipo o un conocido tiene uno. Pasar datos de un chiplet a otro cuesta varias veces más que dentro del mismo, así que el castigo por distancia se parecería más al de varios sockets. Hay que medirlo antes de darlo por hecho.

En 20 segundos: «Tienes razón: no tenemos 4 sockets ni NUMA, y por eso no sacamos conclusiones sobre eso. Comparamos algoritmos entre sí, con los hilos escalados para que 24 sea toda la máquina, igual que 144 en el paper.»

Cargo 2. «Vuestra simulación de sockets es un decorado»

Juez escéptico: Decís que simuláis 4 Xeon con los 4 clústeres de núcleos E. Pero un clúster no es un socket.

Juez defensor: Por eso **descartamos el decorado de verdad**. Una máquina virtual con 4 nodos NUMA engaña al sistema operativo, pero ningún viaje tarda más (Parte 5.1). En su

lugar:

1. **Medimos antes de suponer.** Una «pelota» entre cada par de núcleos dio 35 ns dentro de un clúster, 47 ns entre clústeres y 86 ns entre núcleos P (Parte 5.2). Las distancias existen y son físicas.
2. **Diseñamos un experimento de dosis-respuesta** con 4 hilos: lo único que cambia es la distancia. Al separarlos, Autores pierde entre un 32 % y un 54 % (Parte 6.4).
3. **Tiene un control.** Con n = 10^6, donde no hay pelea por la misma línea de caché, separar no cambia nada: ±2 % con Docker, hasta +3 % sin Docker. Si el montaje estuviera mal, el castigo aparecería también ahí.

Juez escéptico: Pero vuestro castigo es de 1,35× y entre sockets reales ronda 3×.

Juez defensor: Sí, y además ese 3× es un dato indirecto, de máquinas parecidas a la del paper. Por eso el documento dice que la simulación «sirve para ver qué algoritmos son sensibles a la distancia, no para predecir los números de la máquina del paper». Es como un simulador de vuelo de juguete: la física es real, pero más suave.

Presidencia: a favor de la réplica, con matiz. El método es serio: medido, con control y con el límite dicho en voz alta. Lo que no se puede hacer es usarlo para predecir cifras de 4 sockets.

Cómo blindarlo: lo mismo que en el cargo 1. Además, enseña la figura `afinidad_dosis_respuesta.png` y señala el panel n = 10^6 plano: es el mejor argumento visual.

En 20 segundos: «No usamos una máquina virtual, porque ahí nada tarda más. Medimos distancias reales, de 35 a 86 ns, y comprobamos con un control que el efecto solo aparece donde hay contención. Es más suave que 4 sockets reales, y lo decimos.»

Cargo 3. «Recortasteis las mediciones»

Juez escéptico: El paper usa la mediana de 7 ejecuciones de 10 s. Vosotros, 5 de 2 s. Habéis medido una quinta parte.

Juez defensor: Lo hicimos para que la figura 5 pasara de unas 5 horas a poco más de una, y lo compensamos: cada punto va en un proceso nuevo y **en orden aleatorio**, para que el calor o una interrupción afecte a todos por igual (Parte 4.3). Después **comprobamos cuánto movía el recorte**, repitiendo con la receta exacta (Parte 6.7):

- Con Docker, en 2 escenarios: desviaciones de hasta un 21 %.
- Sin Docker, en los 6 escenarios (30 puntos): de −10 % a +19 %. La mitad, menos de un 6,7 %.
- **El orden se mantiene en 4 de 6.** En los otros dos, los que se cruzan están a menos de un 4 %, o en un empate que ya se conocía.

Juez escéptico: Un 19 % no es poco. Y encima decís que hay un sesgo.

Juez defensor: Lo decimos nosotros mismos: con contención, la versión corta da **más** en 23 de 25 puntos, y con n = 10^6 da **menos** en los 5. Por eso la regla del documento es que **una diferencia pequeña no es una victoria**: solo cuentan las de 2 o 3 veces. Las conclusiones de la Parte 7 se apoyan en diferencias de ese tamaño, o se declaran empate.

Presidencia: a favor de la réplica, con matiz. El recorte está validado y sus consecuencias, aplicadas: la réplica no presume de diferencias pequeñas. El sesgo sin explicar es un punto débil real, pero declarado.

Cómo blindarlo: medir la figura 5 completa con la receta exacta, unas 5 horas, una noche. El experimento `paridad` de `baterias.py` ya mide con 10 s × 7; habría que ampliarlo a todos los hilos.

En 20 segundos: «Recortamos para que cupiera en una noche, y luego medimos el coste: el orden se mantiene en 4 de 6 escenarios, y en los otros 2 son empates. Por eso no damos por buena ninguna diferencia pequeña.»

Cargo 4. «Folly no se reproduce porque lo hicisteis mal»

Juez escéptico: En el paper, con contención, Folly rinde entre 0,05× y 0,07× de Autores: veinte veces menos. A vosotros os sale entre 0,37× y 1,37×, y en un escenario gana. Lo más probable es que lo compilarais mal.

Juez defensor: Es la primera sospecha que tuvimos, y la comprobamos de cuatro formas:

1. **Es su Folly.** Sale del artefacto oficial de los autores (Zenodo, MD5 comprobado), compilado con su propio guion dentro de Docker.
2. **Compilado de dos formas, sale igual.** Dentro de Docker con `getdeps.py`, y fuera con CMake directo: mediana de 1,00× entre los dos.
3. **No es el recorte.** Con la receta exacta (7 × 10 s): de 0,36× a 1,26×.
4. **La prueba clave: con un solo hilo.** Ahí no hay pelea entre hilos ni nada viaja entre núcleos. En el paper, Folly ya rinde 0,20× con un hilo; aquí, 0,86×. En cambio, **Williams con un hilo coincide** con el paper (0,36× allí y 0,30× aquí, por ejemplo). El programa de pruebas funciona bien; lo distinto es Folly en la máquina del paper.

Y una pista del propio paper: en su figura 6, medida en otra máquina, el `load` de Folly sin contención tarda 24 ns, casi como Autores (23 ns). Folly no es lento de nacimiento.

Juez escéptico: Muy bien, pero no sabéis la causa.

Juez defensor: No, y no la inventamos. Hay candidatos: la generación del chip, o que la memoria estaba repartida entre 4 sockets incluso con un hilo. Pero con este PC no se puede saber, y el documento lo deja como pregunta abierta.

Presidencia: a favor de la réplica. La acusación era «lo hicisteis mal», y la defensa la desmonta con un argumento muy bueno: la brecha aparece con un hilo, mientras Williams coincide. Que la causa siga abierta no es un fallo de la réplica; es un hallazgo.

Cómo blindarlo: - **Escribir a los autores** preguntando qué versión y configuración de Folly usaron en la figura 5. Cuesta un correo y puede cerrar la pregunta. - Comprobar qué versión de Folly trae el artefacto y si coincide con la del paper. - Medir la latencia de un solo `load` de Folly con un hilo, con contadores de hardware. Para eso habría que instalar `perf`, que este núcleo no trae.

En 20 segundos: «Lo primero que sospechamos fue un error nuestro, y lo descartamos. Es el Folly del artefacto, sale igual compilado de dos formas y con la receta exacta. Y la diferencia

ya existe con un solo hilo, donde Williams sí coincide con el paper. La causa está en algo de su máquina que no podemos reproducir.»

Cargo 5. «La latencia es puro ruido»

Juez escéptico: Con contención, vuestra latencia cambia muchísimo de una ronda a otra. Con 15 rondas y p = 24, la ronda más lenta casi sextuplica a la más rápida (5,75× de mediana, 36,9× en el peor caso). Con eso no se puede concluir nada.

Juez defensor: Con contención, poco, y lo reflejamos en el veredicto:

- **Subimos de 5 a 15 rondas** precisamente por eso (Parte 6.2).
- **Bajamos lo que no aguantó.** La fila 11 pasó de «Se reproduce» a «No se confirma»: la mejora se quedó en un 1-4 %, dentro del ruido. Las filas 7 y 9 se sostienen «por poco», y la 12, «en parte».
- **Tenemos una pista de dónde sale parte del ruido.** Con p = 24, el `load` de Autores sale en dos grupos, unos 420 ns y unos 1 450 ns. En las 3 rondas con los hilos fijados, el segundo grupo no apareció. Es un indicio de que depende de dónde coloca Linux al hilo medido, no una prueba.
- **p = 56 no se usa como comparación.** Con 56 hilos en 24 núcleos hay más hilos que núcleos, así que se mide, pero se declara no comparable.

Sin contención (p = 1), la latencia es estable: la ronda más alta y la más baja coinciden (1,00× en el caso típico y 1,1× en el peor, tabla «Variabilidad de la latencia entre rondas»). Ahí las filas 8 y 10 son claras.

Presidencia: concedido en parte. La latencia con contención es la parte más débil de la réplica, pero el documento lo trata bien: no presume de diferencias pequeñas y rebajó una fila cuando los datos lo pidieron.

Cómo blindarlo: - Medir la figura 6 con la **frecuencia fija** (sin turbo), **núcleos aislados** del sistema y el **hilo medido siempre en el mismo núcleo**. Eso debería estrechar los rangos. - Dar los percentiles con intervalos de confianza (ver cargo 11).

En 20 segundos: «Con contención, la latencia es muy variable. Por eso subimos a 15 rondas, y cuando una fila no aguantó la bajamos a “No se confirma”. Sin contención, los resultados son estables.»

Cargo 6. «Docker contamina las medidas»

Juez escéptico: Medisteis dentro de un contenedor. Eso añade capas, y no sabéis cuánto distorsiona.

Juez defensor: Sí lo sabemos, porque lo medimos: los mismos programas oficiales, dentro y fuera de Docker. **De 270 puntos de la figura 5, 259 cambian menos de un 10 %**, y 236, menos de un 5 %. Las medianas por algoritmo van de 0,99× a 1,01× (Parte 6.6).

Juez escéptico: Pero dentro y fuera cambian más cosas que Docker.

Juez defensor: Cierto: g++ 12.3 frente a 12.4, glibc 2.35 frente a 2.39, Boost 1.74 frente a 1.83, jemalloc 5.2.1 frente a 5.3.0 y la forma de compilar Folly. Por eso, cuando algo cambia,

como Williams entre clústeres, **no lo achacamos a Docker sin más** (Parte 8).

Presidencia: a favor de la réplica. Medido, no supuesto, y con las salvedades bien dichas.

Cómo blindarlo: ya está blindado. Enseña la figura `docker_frente_a_nativo_oficial.png`: casi todos los puntos caen sobre la diagonal.

En 20 segundos: «Lo medimos: de 270 puntos, 259 cambian menos de un 10 % dentro y fuera de Docker.»

Cargo 7. «Os equivocasteis tres veces: ¿por qué fiarse del resto?»

Juez escéptico: Vuestro propio documento confiesa tres errores. El programa del equipo, `bench.cpp`, tenía compartición falsa. Medisteis a Williams con un candado escondido. Y la primera medición de RAM dio lo contrario de la buena. ¿Cuántos más hay que no habéis visto?

Juez defensor: Los tres los encontramos nosotros, con controles diseñados para eso, y los tres están documentados:

1. **Compartición falsa en `bench.cpp`.** Sus 10 pilas cabían en 2 líneas de caché, y eso frenaba a Autores hasta 3,65 veces con n = 10. Lo destapó comparar con el programa oficial. Con las pilas alineadas, Autores pasa de 35 a 120 Mop/s, y la batería entera se repitió así (Parte 6.6).
2. **Williams con candado.** El CMake de latencia del artefacto compila sin `-mcx16`, y sin esa bandera la operación de 16 bytes lleva un candado escondido. Lo encontramos, recompilamos y lo comprobamos en el binario: 8 instrucciones `cmpxchg16b` y ninguna llamada al candado. Las medidas malas se conservan aparte, marcadas como descartadas (Parte 6.2).
3. **La RAM.** La primera versión medía mientras el programa liberaba memoria. Lo delató que el tráfico no crecía con el rendimiento (Parte 8).

Además, **el veredicto no se apoya en `bench.cpp`**, sino en los programas oficiales. `bench.cpp` es una segunda opinión.

Juez escéptico: Eso no responde a cuántos quedan.

Juez defensor: Nadie puede responder a eso. Lo que sí podemos decir es que buscamos errores activamente, y que cuando los encontramos los contamos.

Presidencia: a favor de la réplica. Un trabajo que documenta sus propios errores, con los datos descartados a la vista, merece **más** confianza, no menos. Pero el escéptico tiene un punto: la mejor defensa contra los errores que nadie ha visto es que otra persona lo repita.

Cómo blindarlo: - Que **otro miembro del equipo** repita una parte en su propio PC. Ya estaba previsto en el reparto («¿se mantiene en otro equipo?»). - Decidir si se corrige la compartición falsa de `codigo/bench.cpp`. Es una línea: `template <class I> struct alignas(128) Pila`.

En 20 segundos: «Los tres errores los encontramos nosotros con controles, y están documentados con los datos descartados a la vista. Además, el veredicto se apoya en los programas oficiales, no en el nuestro.»